Website Operations & Recovery Manual

How the 7Band Financial Agency website is hosted, how to change it, and exactly what to do if it ever goes down again.

PROPERTY	**www.7bandfinancialagency.com**
HOSTING	GitHub Pages (free)
SOURCE OF TRUTH	github.com/eastmalik/7band-financial
DOMAIN REGISTRAR	Hostinger
DOCUMENT DATE	August 2026

SECTION

Contents

READ THIS FIRST

If your website is down right now, go straight to **Section 4 — Emergency Playbook** on page 8. It is written to be followed under pressure, in order, without reading anything else in this manual.

SECTION 1

How Your Website Works Now

Your website is not one thing. It is four separate services, each doing one job. Understanding which service does what is the difference between a five-minute fix and a day of downtime.

The four pieces

Piece	Who provides it	What it does	If it fails
The domain name	Hostinger	Owns and registers *7bandfinancialagency.com* in your name.	Nobody can reach your site by name. Renew on time.
DNS records	Hostinger	The address book that tells browsers which server holds your site.	Visitors go nowhere, or to the wrong server.
The files	GitHub (repository)	Every page, image and setting your website is made of.	Nothing — this is version-controlled and recoverable.
The hosting	GitHub Pages	The server that actually shows your site to visitors.	Site is down until hosting is restored.

How a visitor reaches your site

Visitor types www.7bandfinancialagency.com → **Hostinger DNS** points the name at GitHub's servers → **GitHub Pages** serves the built website files → **Site loads** over secure HTTPS

How a change reaches your site

This is the part that makes your setup self-maintaining. You never upload files to a server by hand. Instead:

1. A change is committed to the **main** branch of your GitHub repository.
2. GitHub Actions automatically notices the change and runs the build.
3. The built site is published to GitHub Pages.
4. The live site updates — typically within two minutes.

The instructions for this live in your repository at `.github/workflows/deploy.yml`. As long as that file exists and the repository is public (or on a paid GitHub plan), deployment is automatic and requires no action from you.

THE KEY PRINCIPLE

You own every layer of this stack. The domain is registered to you. The files are in your GitHub account. The hosting is your GitHub account. No outside company holds a piece that they could switch off. That is the specific problem this migration was designed to solve — and it is why the failure you experienced cannot happen the same way twice.

SECTION 2

What Went Wrong — and Why

Understanding the failure is what prevents the next one. The short version: your website was built on a platform that held both the hosting and the images, and when that account was deleted, both disappeared at the same moment.

The sequence of events

1. The website was originally built on **Manus**, an AI website-building platform.
2. Manus hosted the live site. Your DNS pointed at their server (`cname.manus.space`).
3. Manus also hosted your images on their private storage, which the site loaded from a special address (`/manus-storage/...`).
4. Your *code* was correctly backed up to GitHub — that part was done right.
5. The Manus account was deleted. The hosting stopped answering, and the images stopped existing.
6. Your DNS was still pointing at a server that no longer served anything. The site went dark, and stayed dark.

> **THE ROOT CAUSE, IN ONE SENTENCE**
>
> Having your code on GitHub is **not** the same as having your website on GitHub — the code was safe, but the hosting and the images lived in someone else's account, and that is what took the business offline.

The three lessons

Lesson one — code backup is not site backup

A repository full of source code cannot serve a website by itself. Something has to build and host it. Ask of any platform: *if this company disappeared tonight, would my site still be up tomorrow?* If the answer is no, you are renting your uptime from a stranger.

Lesson two — never let assets live off-repository

Your logo, portrait and background images were stored on Manus's cloud rather than inside your own project. When the account went, they went. Images, PDFs, videos and any file your site displays should be committed into the repository itself, where they are versioned, backed up and portable. This has now been done for your logo and portrait.

Lesson three — know your DNS before you need it

Recovery was blocked not by anything technical but by uncertainty over which DNS records to change. Two records were pointing at the dead platform; everything else — your email, in particular — had to be left untouched. Section 7 of this manual records exactly which records matter, so that decision never has to be made under pressure again.

WHAT WAS PERMANENTLY LOST

Two background images could not be recovered: the homepage hero background and the "victory tree" section background. The site was reviewed without them and the design holds up — the deep black and gold treatment reads as intentional. They can be replaced at any time by adding files named `7band-hero-bg-v2_c2aa05ef.jpg` and `7band-victory-tree_5ae0c46c.jpg` to the image folder.

SECTION 3

The Migration, Step by Step

This is the complete record of what was done to bring the site back online, using your real values. If this ever has to be repeated — for this site or a different one — this section is the recipe.

Stage 1 — Strip out the dead platform

The project contained code that only functioned inside Manus's environment: a runtime plugin, a debug log collector, a storage proxy, and an analytics tracker injected into every page. None of it could work any more, and the analytics tag was actively broken. All of it was removed, and the build configuration was reduced to what the site genuinely needs.

Stage 2 — Set up automatic hosting on GitHub Pages

A deployment workflow was added at `.github/workflows/deploy.yml`. On every change to the **main** branch it installs dependencies, builds the site, adds a fallback page so that direct links to interior pages work correctly, and publishes the result to GitHub Pages.

Stage 3 — Point the domain at GitHub

Two files and two DNS records were involved. In the repository, a `CNAME` file was added containing the domain name, which is how GitHub Pages knows which custom domain to answer for. At Hostinger, two records that still pointed at Manus were repointed:

Record type	Name	Old value (dead)	New value (correct)
CNAME	www	`cname.manus.space`	`eastmalik.github.io`
ALIAS	@	`cname.manus.space`	`eastmalik.github.io`

IMPORTANT — DO NOT ADD GITHUB'S IP ADDRESSES

Standard GitHub Pages instructions tell you to create four **A records** pointing at IP addresses (185.199.108.153 and similar). **Do not do this on your domain.** Hostinger provides an **ALIAS** record instead, which does the same job and automatically follows GitHub if they ever change those addresses. Adding A records alongside the existing ALIAS record would create a conflict. Your DNS is correct as it stands.

Stage 4 — Activate GitHub Pages and HTTPS

In the repository's **Settings** → **Pages** screen, the source was set to **GitHub Actions** and the custom domain entered. GitHub then ran its own DNS check and issued a free security certificate. Note that the very first activation must be done by hand in that settings screen — an automated workflow cannot create a Pages site from nothing.

Stage 5 — Restore the images

The company logo and portrait were uploaded directly into the repository at `client/public/manus-storage/` and renamed to the exact filenames the site expects. A favicon set was generated from the logo so the gold mark appears in browser tabs, bookmarks and phone home screens.

REQUIRED IMAGE FILENAMES

The site looks for these exact names inside `client/public/manus-storage/`:

```
7band-logo-clean_7b539e21.png   — company logo (restored)
malik-east-portrait_1eb03c6e.jpeg — About page portrait (restored)
7band-hero-bg-v2_c2aa05ef.jpg   — homepage hero background (not recovered)
7band-victory-tree_5ae0c46c.jpg — victory section background (not recovered)
```

SECTION 4

Emergency Playbook

The site is down. Work through these checks in order and stop as soon as one of them explains the problem. Most outages are answered within the first three checks.

BEFORE YOU DO ANYTHING

Confirm the site is actually down. Open it in a private/incognito window, and on a phone using mobile data rather than your office wi-fi. Browsers cache pages aggressively, and more than one "outage" has turned out to be one stale browser tab. If it loads anywhere, the site is up.

CHECK 1 Is the deployment failing?

Open **github.com/eastmalik/7band-financial/actions**. Look at the most recent run. A green check means the site was published successfully and the problem is elsewhere — move to Check 2. A red X means the build failed: click into it, read the error, and the previous working version is still live in the meantime.

CHECK 2 Is GitHub Pages itself switched on?

Open **Settings** → **Pages** in the repository. Source must read **GitHub Actions** and the custom domain must read **www.7bandfinancialagency.com**. If the custom domain field has been emptied — this can happen after certain repository changes — retype it and save.

CHECK 3 Is DNS still pointing at GitHub?

In Hostinger, open **Domains** → **DNS / Nameservers**. The **www** CNAME record and the @ ALIAS record must both read **eastmalik.github.io**. If either has changed or been deleted, restore it. Allow up to a few hours for the change to spread worldwide.

CHECK 4 Has the domain expired?

In Hostinger, check the domain's renewal date. An expired domain takes down the website and the email at the same time — if email is also dead, check this first. Renewal restores service within hours.

CHECK 5 **Is GitHub having an outage?**

Open **githubstatus.com**. If GitHub Pages is reported as degraded, nothing on your side is broken and nothing you change will help. Wait it out, and tell customers you are aware of a hosting provider issue.

If the site is up but something looks wrong

Symptom	Most likely cause	What to do
A broken-image icon where a picture should be	The image file is missing from the repository, or its filename does not match what the page asks for.	Upload the file to client/public/manus-storage/ with the exact expected name.
Your change did not appear	The deployment is still running, or your browser is showing a cached copy.	Wait two minutes, then hard-refresh (Ctrl+Shift+R). Check the Actions tab.
Browser shows a security warning	The HTTPS certificate is still being issued, or the domain was recently changed.	Wait — it resolves itself. Confirm Enforce HTTPS is ticked in Settings → Pages.
An interior page 404s when opened directly	The SPA fallback file was not created during the build.	Confirm the 'Add SPA fallback' step still exists in deploy.yml.
Old browser tab icon still showing	Favicons are cached harder than anything else on the web.	Close the tab completely and reopen, or check on a different device.

SECTION 5

Backup & Protection Strategy

The migration restored your site. This section is about making the outage unrepeatable — and closing the one gap that remains.

Where you stand today

Asset	Protection status
Website code	**Protected.** Full version history in GitHub; every past version recoverable.
Logo and portrait	**Protected.** Now committed into the repository, not an outside cloud.
Hosting	**Protected.** Runs in your own GitHub account; no third-party platform involved.
Domain name	**Protected** while renewal is current. Enable auto-renew at Hostinger.
Business email	**Unchanged.** Still handled by Hostinger and Mailgun; untouched by this migration.
Repository privacy	**Open gap.** The repository is public — see below.

Closing the privacy gap

Free GitHub Pages hosting requires a **public** repository. Your website is currently public, which means the source code can be browsed by anyone who finds it. In practical terms this exposes the same text, images and links that any visitor already sees on the site — but the preference for privacy is legitimate, and there are two ways to satisfy it.

Option A — GitHub Pro

Approximately $4 per month.

Makes the repository private while the website stays publicly visible. Nobody can browse your code; customers see the site normally. Nothing about your deployment changes.

Recommended. For a business of your size this is the obvious choice — it removes the gap entirely for the price of a coffee.

Option B — Private mirror

Free.

The deployment repository stays public, and a second private repository holds a synchronised backup copy. Your code remains publicly visible in the first one.

Fallback only. This adds a backup, but it does not deliver the privacy you asked for.

Standing rules to operate by

- **Every image the site uses belongs in the repository.** Never link a live page to an image hosted inside another company's platform. This single rule would have prevented most of what you lost.
- **Turn on auto-renew for the domain.** An expired domain takes down the website and the email together, and recovery is slower than you would expect.
- **Keep two-factor authentication on the GitHub account.** It is now the account your website's existence depends on. Store the recovery codes somewhere physical.
- **Never delete an account that hosts anything live.** Move the service first, confirm the replacement works for several days, then close the old account.
- **Before any provider change, ask one question:** if this company vanished tonight, what breaks? Write the answer down before you proceed.

A NOTE ON RECOVERY TIME

Every past version of your website is stored permanently in GitHub's history. If a future change breaks something, the site can be restored to any earlier state in minutes, not hours. This is the practical benefit of the setup you now have, and it did not exist before this migration.

SECTION 6

Making Changes to Your Site

Your website is now edited by describing what you want in plain English. No software to install, nothing to upload, no server to log into.

How the workflow actually runs

1. You describe the change you want — in ordinary words, not technical instructions.
2. The change is made in the code and verified before anything goes live.
3. It is committed to the **main** branch of your repository.
4. GitHub automatically builds and publishes it.
5. The live site reflects the change, typically within two minutes.

What can be changed this way

- Any text on any page — headlines, body copy, button labels, contact details
- Links, including booking and calendar destinations
- Images: replacing, adding or removing them
- Colours, fonts, spacing and layout
- Whole new pages, or removing pages you no longer offer
- Page titles and search-engine descriptions
- Navigation menus and their ordering

Asking for changes effectively

You do not need technical vocabulary. Describe the outcome you want and, where it helps, say where on the site you mean.

Instead of worrying about	Just say
Which file, which component, what the code is called	**"Change the phone number in the footer to 555-0142"**
How images are referenced and sized	**"Swap the photo on the About page for the new one I uploaded"**
Routing, navigation and page structure	**"Add a Services page with these three offerings"**

Styling systems and colour values	**"Make the Begin Your Quest button larger and easier to see"**

Adding image files

The most reliable way to get an image into the site is to upload it directly to GitHub, which also guarantees it is backed up from the moment it arrives:

1. Go to your repository and open the folder `client/public/manus-storage/`.
2. Choose **Add file** → **Upload files** and drag the image in.
3. Click **Commit changes**.
4. Say which image is which and where it should appear — filenames do not need to be correct, they can be renamed afterwards.

IF YOU ARE WORKING WITH A DIFFERENT ASSISTANT

Any capable developer or AI assistant can maintain this site, but they will need the context that this manual contains. Give them three facts: the site is a **React and Vite** application, it deploys automatically to **GitHub Pages** from the **main** branch via the workflow in `.github/workflows/deploy.yml`, and images live in `client/public/manus-storage/`. Those three facts plus the appendix opposite are enough for anyone competent to work safely.

SECTION 7

Security Posture

A full security audit of the website was carried out in September 2026. This section records what was checked, what makes the site hard to attack, and where the real risk actually sits.

Why this site is a hard target

The website has **no forms, no logins, no database, and no user input of any kind**. It is a set of static files served by GitHub. That single fact removes most categories of website attack outright:

- **Database attacks** — impossible; there is no database.
- **Password and login attacks** — impossible; there is nothing to log into.
- **Form spam and injection** — impossible; the site has no forms.
- **Server compromise** — there is no server to compromise; GitHub serves flat files.
- **Malicious file upload** — impossible; nothing accepts uploads.

The migration away from Manus happened to produce one of the most secure architectures a business website can have.

What the audit checked

Check	Result
Secrets or API keys in the code	None found, across the full commit history — not just current files.
Secrets leaking into public JavaScript	None.
Credential or key files ever committed	None.
Cross-site scripting (XSS) patterns	None on any live page.
Malicious link hijacking (tabnabbing)	Protected — every external link is correctly secured.
Source maps exposing internal code	Not published.
Deployment pipeline permissions	Correctly minimal: read-only except for publishing.
Data stored on visitors' devices	View preference and theme only. No personal data.
Web address parameter handling	Safe — validated against a fixed list, never inserted into the page.
Dependency vulnerabilities	128 advisories, all in build tooling. None reach visitors.

Hardening that was applied

- **Clickjacking protection.** GitHub Pages cannot send the security headers that normally stop a site being wrapped inside someone else's page, so a guard script was added instead. If anyone embeds this site in a frame, the wrapper is replaced by the real site. Verified working.
- **Strict referrer policy**, so the site does not leak full page addresses to third parties.
- **Dead Manus code removed.** An unused component read an API key from a build variable. Nothing was ever set there, but had it been, the value would have been compiled into the public JavaScript for anyone to read. That trap is now gone, along with unused components still pointing at retired Calendly booking links.

WHERE THE REAL RISK SITS — READ THIS PART

Nobody is going to hack the web pages. An attacker would go after the **accounts that control them**:

1. The GitHub account. Whoever controls it controls the website. This is now the single most critical login in the business.
2. The Hostinger account. Controls the domain, the DNS, and the business email. Access here could redirect the entire domain.
3. The email inbox. It is the password-reset path to everything else.

Strong unique passwords plus two-factor authentication on those three accounts handles the overwhelming majority of genuine risk to this business. No amount of work on the website itself substitutes for it.

Ongoing habits

- Keep two-factor authentication switched on for GitHub, Hostinger and email, and store the recovery codes somewhere physical.
- Never paste an API key, token or password into a chat window — including with an AI assistant. Credentials belong in a password manager.
- Treat any email asking you to “verify” a GitHub or Hostinger login as hostile until proven otherwise. Type the address in yourself rather than clicking the link.
- If a change to the site ever looks wrong or unfamiliar, the full version history is in GitHub and any earlier version can be restored in minutes.
- Re-run this audit after any major change to how the site is built or hosted.

SECTION A

Appendix: Reference Card

Every value someone would need in an emergency, in one place. Print this page and keep it somewhere that does not depend on your website being up.

Live addresses

What	Where
Live website	`https://www.7bandfinancialagency.com`
Code repository	`https://github.com/eastmalik/7band-financial`
Deployment history	`https://github.com/eastmalik/7band-financial/actions`
Hosting settings	`https://github.com/eastmalik/7band-financial/settings/pages`
Image upload folder	`github.com/eastmalik/7band-financial → client/public/manus-storage`
Domain & DNS control	`Hostinger → Domains → DNS / Nameservers`
GitHub service status	`https://githubstatus.com`

DNS records that must not change

Type	Name	Value	Purpose
CNAME	www	`eastmalik.github.io`	Points www at the website
ALIAS	@	`eastmalik.github.io`	Points the bare domain at the website

DO NOT TOUCH THE EMAIL RECORDS

Every other record in your Hostinger DNS — the MX records, the hostingermail entries, the mailgun and leadconnectorhq entries, the DKIM, SPF and DMARC text records — exists to keep your business email working. They have nothing to do with the website. Changing or deleting them will break email delivery.

Key files in the repository

Path	What it controls
`.github/workflows/deploy.yml`	The automatic build-and-publish process. Without this, changes never go live.
`client/public/CNAME`	Tells GitHub Pages which custom domain to answer for.
`client/index.html`	Page title, search description, and the browser-tab icon.
`client/src/pages/`	One file per page of the website.
`client/public/manus-storage/`	Site images. The folder name is historical; it is now just a folder.
`client/public/favicon.ico`	The browser-tab icon, generated from the company logo.

Accounts this website depends on

Record where each of these logins is stored — a password manager, a safe, wherever your practice is. Do not write the passwords themselves on this page.

Account	Controls	Login stored
GitHub (eastmalik)	Website files and hosting. The critical one.	
Hostinger	Domain registration, DNS, business email.	
Mailgun / LeadConnector	Email delivery, referenced in your DNS.	

The one thing to remember

Your website's files and its hosting now live in the same account — an account you own and control. No outside company can switch it off. Keep the domain renewed, keep the GitHub account secure, and keep every image inside the repository, and this site stays up.